

Trilha de Node.js

*Módulo 06: Autenticação e
Segurança*

Instruções para a melhor prática de Estudo

1. **Leia atentamente todo o conteúdo:** Antes de iniciar qualquer atividade, faça uma leitura detalhada do material fornecido na trilha, compreendendo os conceitos e os exemplos apresentados.
2. **Não se limite ao material da trilha:** Utilize o material da trilha como base, mas busque outros materiais de apoio, como livros, artigos acadêmicos, vídeos, e blogs especializados. Isso enriquecerá o entendimento sobre o tema.
3. **Explore a literatura:** Consulte livros e publicações reconhecidas na área, buscando expandir seu conhecimento além do que foi apresentado. A literatura acadêmica oferece uma base sólida para a compreensão de temas complexos.
4. **Realize todas as atividades propostas:** Conclua cada uma das atividades práticas e teóricas, garantindo que você esteja aplicando o conhecimento adquirido de maneira ativa.
5. **Evite o uso de Inteligência Artificial para resolução de atividades:** Utilize suas próprias habilidades e conhecimentos para resolver os exercícios. O aprendizado vem do esforço e da prática.
6. **Participe de debates:** Discuta os conteúdos estudados com professores, colegas e profissionais da área. O debate enriquece o entendimento e permite a troca de diferentes pontos de vista.
7. **Pratique regularmente:** Não deixe as atividades para a última hora. Pratique diariamente e revise o conteúdo com frequência para consolidar o aprendizado.
8. **Peça feedback:** Solicite o retorno dos professores sobre suas atividades e participe de discussões sobre os erros e acertos, utilizando o feedback para aprimorar suas habilidades.

Essas instruções são fundamentais para garantir um aprendizado profundo e eficaz ao longo das trilhas.

Módulo 6: Autenticação e Segurança

Objetivo:

Garantir a segurança nas aplicações web, protegendo os dados dos usuários por meio de boas práticas de desenvolvimento, autenticação segura e prevenção contra vulnerabilidades comuns.

6.1: Introdução à autenticação com JWT (JSON Web Token)

O que é JWT?

1. **Definição:** Um padrão aberto para transmissão segura de informações entre partes como um objeto JSON compactado.
2. **Composição:**
 - a. **Header:** Define o tipo de token e o algoritmo de assinatura.
 - b. **Payload:** Contém as informações (claims) codificadas.
 - c. **Signature:** Garante a integridade dos dados.

Como funciona?

- O cliente envia credenciais (ex.: login e senha).
- O servidor valida e emite um token JWT.
- O cliente armazena o token e o envia em cada requisição subsequente.

Exemplo de Uso:

```
const jwt = require('jsonwebtoken');

const token = jwt.sign({ id: 1, role: 'admin' }, 'secret_key', { expiresIn: '1h' });
console.log(token);

const decoded = jwt.verify(token, 'secret_key');
console.log(decoded);
```

6.2: Gerenciamento de Sessões e Cookies

Sessões

1. Armazenam dados temporários relacionados ao usuário no servidor.
2. Usado para persistir estados entre requisições HTTP.

Cookies

1. Pequenos arquivos armazenados no navegador.
2. Contêm informações como tokens de autenticação.

Exemplo de Uso no Express:

```
const session = require('express-session');
const cookieParser = require('cookie-parser');

app.use(cookieParser());

app.use(session({
  secret: 'chave_secreta',
  resave: false,
  saveUninitialized: true
}));
```

Dicas de Segurança:

- Configure o atributo **HttpOnly** para evitar acessos via JavaScript.
 - Use **Secure** para transmitir cookies apenas via HTTPS.
-

6.3: Proteção contra vulnerabilidades comuns (CORS, CSRF, SQL Injection)

CORS (Cross-Origin Resource Sharing)

- **Definição:** Um mecanismo que controla como recursos de um servidor podem ser acessados a partir de domínios diferentes.
- **Problema Resolvido:** Bloqueia requisições não autorizadas de origens externas.
- **Configuração no Express:**

```
const cors = require('cors');

app.use(cors({
  origin: 'https://dominio-autorizado.com', methods: ['GET', 'POST']
}));
```

CSRF (Cross-Site Request Forgery)

- **Definição:** Ataque onde comandos maliciosos são enviados de um usuário autenticado para um servidor.
- **Solução:**
 - Utilize tokens CSRF que são validados pelo servidor.
 - Exemplo no Express:

```
const csrf = require('csrf');

app.use(csrf());

app.get('/form', (req, res) => {
  res.render('form', { csrfToken: req.csrfToken() });
});
```

SQL Injection

1. **Definição:** Ataque que explora falhas nas consultas SQL, permitindo a injeção de comandos maliciosos.
2. **Exemplo de Vulnerabilidade:**

```
SELECT * FROM usuarios WHERE nome = 'admin' OR '1'='1';
```

- **Solução:**
 - Use prepared statements:

```
connection.query(
  'SELECT * FROM usuarios WHERE nome = ?', [nome],
  (err, results) => {
    if(err) throw err;
    console.log(results);
  }
);
```

Aula 6.4: Implementando autenticação baseada em roles com MySQL

O que é Autenticação Baseada em Roles?

- Permite restringir ações com base nos papéis (roles) dos usuários, como **admin**, **editor** e **usuário comum**.

Exemplo de Estrutura de Tabelas:

```
CREATE TABLE usuarios (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  email VARCHAR(120) NOT NULL UNIQUE,
  senha VARCHAR(60) NOT NULL,
  role ENUM('admin', 'user') NOT NULL DEFAULT 'user'
);
```

Middleware de Autorização:

```
function authorize(role) {
  return(req, res, next) => {
    if(req.user.role !== role) {
      return res.status(403).send('Acesso negado');
    }

    next();
  };
}

app.get('/admin', authorize('admin'), (req, res) => {
  res.send('Bem-vindo, Admin!');
});
```

 **Observação:** Consulte aqui o material de exemplo com práticas para treinar suas habilidades: [Módulo 6](#)

Lista de Exercícios

Questões Teóricas

- Explique o que é JWT e quais são suas vantagens.
- Qual a diferença entre sessões e cookies?
- Defina CORS e por que é importante configurá-lo corretamente.
- O que é CSRF e como prevenir este tipo de ataque?
- Explique o conceito de SQL Injection e seus riscos.
- Como o atributo `HttpOnly` ajuda a proteger cookies?
- Quais são as principais partes de um token JWT?
- Liste três boas práticas para evitar SQL Injection em aplicações Node.js.
- O que é autenticação baseada em roles e quais são seus benefícios?
- Como o middleware `csrf` auxilia na proteção de aplicações web?

Questões Práticas

- Configure um servidor Express para gerar e validar tokens JWT.
- Implemente um sistema de sessão que armazene informações temporárias de um usuário.
- Configure o CORS para permitir apenas requisições de um domínio específico.
- Crie uma rota protegida que utilize tokens CSRF para validação.
- Desenvolva uma função que previna SQL Injection utilizando prepared statements.
- Configure cookies seguros com os atributos `HttpOnly` e `Secure`.
- Crie uma tabela de usuários com roles e insira dados fictícios.
- Implemente middleware para restringir acesso com base em roles.

- Simule um ataque CSRF e implemente a solução para preveni-lo.
- Crie um sistema de login simples que emita um JWT e proteja rotas baseadas no token.